

SZENARIO_1F_Record_ETL_file

Szenario 1F : Record-ETL und Qualitätsprüfung (file-only, ohne DB)

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login admin N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0 Fertiges Referenz-Lab auf der Canvas: “Szenario 1F - Record ETL & Qualitaet (file)” Live gebaut und verifiziert. Kein DBCPConnectionPool / SQLite nötig.

Unterschied zur DB-Variante (SZENARIO_1_Record_ETL.md): PutDatabaseRecord ist durch PutFile ersetzt. Schema-Validierung, SQL-Filter und Quarantäne sind **identisch**. Der “saubere Pfad” landet als Datei statt in der DB.

Flow:

```
GetFile → ValidateRecord → QueryRecord → PutFile (sauber)
                        | invalid          | original (terminiert)
                        ▼
                        PutFile (Quarantäne)
```

Das Record-Konzept (Kernidee)

Record-Prozessoren sehen die FlowFile als **Liste von Datensätzen mit Schema**, nicht als Bytes. Dadurch sind Validierung und SQL pro Datensatz möglich. Zwei Controller Services: - **CSVReader** : parst CSV in Records, kennt das Schema. - **CSVRecordSetWriter** : schreibt Records wieder raus.

Vorbereitung

Avro-Schema (Pflichtfelder + Typen, der “Vertrag” für die Validierung):

```
{ "type": "record", "name": "auftrag", "fields": [
  { "name": "auftrag_id", "type": "int" },
  { "name": "kunde", "type": "string" },
  { "name": "betrag", "type": "double" },
  { "name": "prioritaet", "type": "string" } ] }
```

Beispieldaten data/samples/auftraege_mix.csv (4 gültig, 1 Typfehler):

```
auftrag_id,kunde,betrag,prioritaet
1001,Mueller GmbH,149.90,hoch      <- gültig, betrag>=50  -> sauber
1002,Schmidt AG,29.50,normal       <- gültig, betrag<50   -> herausgefiltert
1003,Weber KG,210.00,hoch          <- gültig, betrag>=50  -> sauber
2001,Fischer SE,abc,hoch           <- betrag kein double -> Quarantäne
2002,Klein OHG,75.00,normal        <- gültig, betrag>=50  -> sauber
```

Controller Services (beide aktivieren)

⚙ der Process Group → Tab **Controller Services** → +

CSVReader: - Schema Access Strategy = Use 'Schema Text' Property - Schema Text = das Avro-Schema oben - Treat First Line as Header = true

CSVRecordSetWriter: - Schema Access Strategy = Inherit Record Schema

Warum festes Schema statt Infer Schema? Damit die Typprüfung greift. Nur durch `betrag:double` wird `betrag=abc` als ungültig erkannt. Bei Inferenz würde NiFi es als Text durchwinken.

Prozessoren

1. GetFile

- Input Directory = `$N/data/input/szenario1F`
- File Filter = `.*\.csv`, Keep Source File = `false`, Polling Interval = `2 sec`

2. ValidateRecord (technische Qualität)

- Record Reader = CSVReader, Record Writer = CSVRecordSetWriter
- Schema Access Strategy = `Use Reader's Schema`
- **Strict Type Checking** = `true` (erzwingt Typen : `betrag=abc` wird invalid)
- Allow Extra Fields = `false`
- Relationships: **valid** → QueryRecord, **invalid** → PutFile Quarantäne, failure → terminieren

3. QueryRecord (fachliche Filterung)

- Record Reader = CSVReader, Record Writer = CSVRecordSetWriter
- Dynamische Property: Name `gefiltert` = `SELECT * FROM FLOWFILE WHERE betrag >= 50`
- Relationships: **gefiltert** → PutFile sauber, original → terminieren, failure → terminieren

4. PutFile sauber (ersetzt PutDatabaseRecord)

- Directory = `$N/data/output/szenario1F/sauber`
- Create Missing Directories = `true`, Conflict Resolution = `replace`

5. PutFile Quarantäne

- Directory = `$N/data/output/szenario1F/quarantaene`
 - Create Missing Directories = `true`, Conflict Resolution = `replace`
-

Verdrahtung

Von	Relationship	Nach
GetFile	success	ValidateRecord
ValidateRecord	valid	QueryRecord
ValidateRecord	invalid	PutFile Quarantäne
ValidateRecord	failure	terminieren
QueryRecord	gefiltert	PutFile sauber
QueryRecord	original, failure	terminieren
beide PutFile	success, failure	terminieren

Stolperstein: `original` von QueryRecord ist die **unveränderte ganze Eingabe**, nicht der "Rest". Nicht an die Quarantäne hängen, sonst landen gültige Daten dort.
→ terminieren.

Starten und prüfen

```
cp $N/data/samples/auftraege_mix.csv $N/data/input/szenario1F/  
sleep 8  
echo "--- sauber ---";      cat $N/data/output/szenario1F/sauber/*  
echo "--- quarantaene ---"; cat $N/data/output/szenario1F/quarantaene/*
```

Verifiziertes Ergebnis (im Live-Test bestätigt)

sauber (gültig + betrag $\geq$ 50):

```
auftrag_id,kunde,betrag,prioritaet  
1001,Mueller GmbH,149.9,hoch  
1003,Weber KG,210.0,hoch  
2002,Klein OHG,75.0,normal
```

quarantaene (Typfehler):

```
auftrag_id,kunde,betrag,prioritaet  
2001,Fischer SE,abc,hoch
```

herausgefiltert (weder sauber noch Quarantäne): 1002 Schmidt AG, 29.50 (betrag $<$ 50).

Drei Datenschicksale, wie im DB-Original: 1. **gültig + fachlich relevant** → sauber (statt DB)
2. **gültig, aber gefiltert** → verworfen (QueryRecord-Bedingung) 3. **ungültig (Typ)** → Quarantäne

Reset

```
find $N/data/input/szenario1F $N/data/output/szenario1F/sauber \
    $N/data/output/szenario1F/quarantaene -type f -delete
```

Häufige Stolpersteine

- **Alles geht in invalid** → Schema-Typen passen nicht zu den Daten; Schema Text und Header prüfen.
- **QueryRecord ohne Ausgang** → SQL-Property wird erst durch Anlegen zur Relationship.
- **Reader/Writer nicht enabled** → Record-Prozessoren invalid (⚠), erst ⚡.
- **Gültige Daten in Quarantäne** → `original` von QueryRecord fälschlich verbunden; terminieren.